AI/ML Resume Portfolio Guide

Five runnable, demo-first projects for resume building, GitHub and interviews.

Project	Primary signal	What it demonstrates
CodePilot AI	Agentic software engineering	Autonomous bug resolution: code retrieval → patch → tests → human-review diff.
VisionGuard AI	Computer vision	Defect classification + anomaly localization + confidence/severity through an inference API.
RecSysX	Recommendation systems	Hybrid content + collaborative ranking with explainable score components.
ML Sentinel	MLOps	Training, model metrics, inference, PSI drift monitoring and retraining.
FinSight AI	NLP / retrieval / forecasting	Grounded document retrieval with citations, sentiment and a lightweight time-series forecast.

Recommended resume strategy

- Use 3–4 projects on a one-page resume; keep all five on GitHub.
- For AI/ML Engineer roles: CodePilot AI + ML Sentinel + RecSysX + either VisionGuard or FinSight.
- Pin the five repositories and add a live demo URL/video to the strongest three.
- Use metrics you actually measured. Do not invent model accuracy, users, latency or scale.

CodePilot AI

Best first project for AI Engineer / ML Engineer roles because it combines retrieval, agent workflows and software engineering.

Stack: Python, FastAPI, RAG-style Retrieval, NLP, Code Search, Pytest, Docker, CI/CD

- Built an autonomous software-engineering agent that indexes a repository, retrieves bug-relevant files using vectorized code search, generates minimal patches, and validates changes with automated tests.
- Implemented bug triage, patch reasoning, sandboxed test execution and unified-diff review through a FastAPI workflow with human approval before changes are accepted.
- Designed provider-agnostic LLM/RAG extension points, Docker packaging and CI testing for a production-style engineering workflow.

VisionGuard AI

Shows visual ML product thinking; add a trained YOLO model before claiming YOLO training metrics.

Stack: Python, FastAPI, Computer Vision, Image Processing, PyTorch/YOLO-ready, Docker

- Built a computer-vision inspection service that detects surface defects, localizes anomalous regions and returns confidence/severity scores through a real-time inference API.
- Implemented preprocessing, ROI analysis, bounding-box visualization, upload validation, Docker deployment and automated tests.
- Added pluggable PyTorch/YOLO model adapters so the inference API can be upgraded to trained deep-learning detectors without changing the application layer.

RecSysX

Strong for product ML and large-tech recommender interviews; optional two-tower/FAISS path included.

Stack: Python, Scikit-learn, Collaborative Filtering, Embeddings, FastAPI, PyTorch/FAISS-ready

- Built a hybrid recommendation engine combining content embeddings, collaborative user behavior and popularity signals to generate personalized top-K recommendations.
- Implemented recommendation explainability, seen-item filtering, catalog-coverage metrics, FastAPI serving, Docker deployment and automated tests.
- Added optional PyTorch two-tower and FAISS modules for learned user/item embeddings and scalable approximate-nearest-neighbor retrieval.

ML Sentinel

Separates you from notebook-only candidates by showing model lifecycle and monitoring.

Stack: Python, Scikit-learn, FastAPI, Drift Monitoring, MLflow/Airflow-ready, Docker, GitHub Actions

- Built an end-to-end MLOps service covering preprocessing, training, versioned metrics, online inference, retraining and production-style feature drift monitoring using PSI.
- Exposed model-health and prediction APIs with input validation, automated tests, Docker packaging and GitHub Actions CI.

- Added optional MLflow experiment tracking and Airflow retraining integrations for model registry, scheduled retraining and alerting workflows.

FinSight AI

Shows evidence-grounded research design and multi-component applied ML.

Stack: Python, FastAPI, NLP, RAG-style Retrieval, TF-IDF, Time-Series Forecasting, Docker

- Built a financial-research assistant that indexes documents, retrieves relevant evidence and returns grounded answers with source-level citations.
- Added document sentiment analysis and revenue forecasting, exposing research and forecast workflows through FastAPI and an interactive dashboard.
- Designed a provider-agnostic generation interface and CI-tested architecture so dense embeddings and a stronger LLM generator can be integrated later.

How to present each project

- 20 sec: state the real problem.
- 45–60 sec: draw/explain data flow.
- 2 min: show one deterministic live demo.
- 45 sec: explain one algorithm/model choice and one metric.
- 30 sec: identify a limitation and how you would scale it.

Questions you must be able to answer

- What data enters the system and what preprocessing happens?
- Why this algorithm/model instead of another one?
- What is the most meaningful metric?
- What can fail and how do you detect it?
- How would it change at 100x data/traffic?
- Which part did you personally implement and debug?

Portfolio completion checklist

- Run the project from a fresh terminal.
- Run `pytest -q` and save a screenshot of the passing tests.
- Push the repository to GitHub with a clean README and demo preview.
- Deploy at least 2–3 of the five apps publicly.
- Record a 60–90 second demo video for each deployed project.
- Add one architecture diagram and one metric/evaluation table to each GitHub README.
- For optional YOLO/PyTorch/FAISS/MLflow/Airflow paths, execute them before claiming them as trained/deployed experience.
- Be ready to explain data flow, algorithm choice, model metric, failure mode, scaling plan and your exact implementation contribution.

Final rule

These projects are intentionally built to be runnable without paid APIs. Optional advanced modules are included where useful, but the best resume is an accurate resume. Execute, measure and understand an optional path before turning it into a claim.